Git Matura

rozproszony system kontroli wersji

Marek Małek

Rozkład jazdy

(part-0)

- intro

(part-1)

- setup
- porcelain
- plumbing
- branch
- merge
- rebase
- reset
- reflog
- HEAD

(part-2)

- remote
- GitHub
- push
- pull
- pr
- .gitignore
- conflicts
- misc

part-0

(story-time)

(part-0): intro



Linus Torvalds

- powstał w celu wsparcia pracy na **Linuxem**
- A git origin story

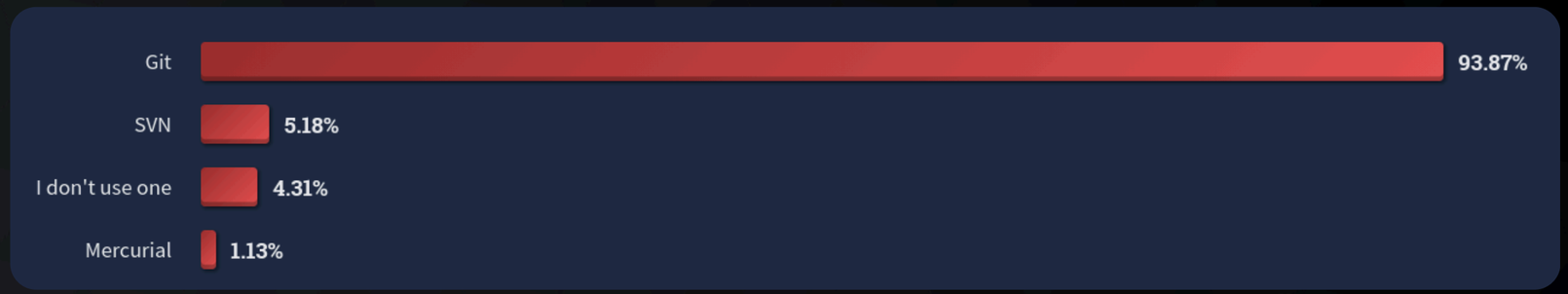








- stał się **standardem** w branży



StackOverflow 2022 survey

(part-0): intro

Jakie problemy rozwiązuje?

- Utrata pracy (np. przypadkowe usunięcie pliku)
- Brak wiedzy, co się zmieniło (“co to tu robi?”)
- Strach przed zepsuciem projektu
- Trudne do znalezienia błędy (“wcześniej działało”)
- Brak „pamięci projektu” (“tamten pomysł był lepszy”)
- (+remote) Problemy przy pracy zespołowej



part-1

(local)

(part-1): setup

Od czego zacząć?

- dokumentacja
- **instalacja**
  - zależy od systemu
  - warto zwerykować np.:

```
git version
git version 2.52.0
```

- manual

```
man git
```

Jak to działa?

- składnia poleceń:

```
command <required> [optional]
```

  - command → git
  - <required> → argument obowiązkowy
  - [optional] → flagi, opcje

(part-1): setup

Konfiguracja `~/.gitconfig`

- żeby było wiadomo kto coś zrobił

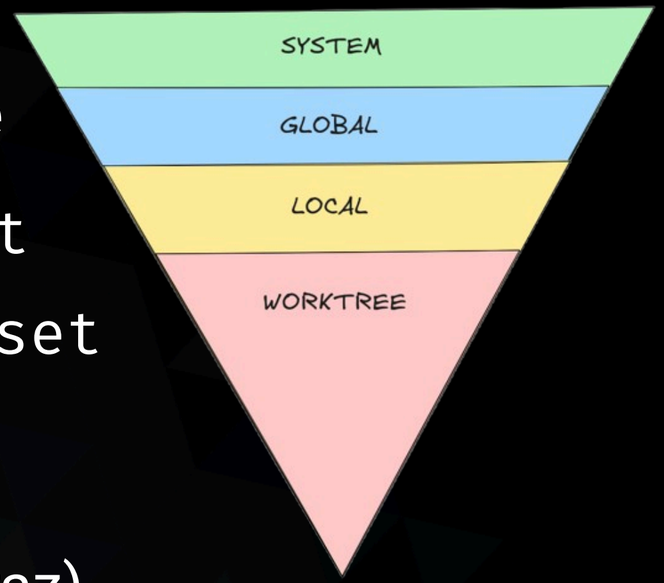
```
git config
```

- sprawdzenie aktualnej konfiguracji

```
git config --list
```

- również przydatne

```
git config get
git config unset
```



- warto ustawić swoją identyfikację globalnie (bo trzeba tylko raz)

```
git config set --global user.name “bitmatura”
git config set --global user.email “bitmatura@proton.me”
```

(part-1): porcelain

Repozytorium

- zasadniczo to po prostu katalog (folder)
  - ale posiadający w sobie katalog `.git` gdzie dzieje się cała magia

```
git init [<directory>]
```

albo

```
mkdir <directory>
cd <directory>
git init
```



(part-1): porcelain

Status

- stan pliku wg. gita
  - `untracked`
  - `staged`
  - `committed`
- stan plików w repozytorium:
  `git status`

Staging

- wskazanie, aby git uwzględnił plik przy zapisie zmian
  - bez tego jest on “tylko u nas”
  - inaczej “index”
- `untracked → staged`
  `git add <path-to-file> | pattern`

(part-1): porcelain

Commit

- “zdjęcie” (snapshot) repozytorium w danym czasie
- ma wiadomość (commit message)
- identyfikowany przez **unikalny hash**
- `staged → committed`
- składnia:

```
git commit -m <msg>
```



Jesteśmy w połowie wykładu

(serio)

(part-1): porcelain

Log

- pokazuje historię commitów
- pozwala na kontrolę wersji
  - co zostało zmienione
  - kto coś zmienił
  - kiedy coś zostało zmienione
- składnia:

```
git log
```

Co wpływa na hash?

- nazwa commita
- nazwa i email autora
- data
- poprzednie hashe

Hashowanie

- SHA-1

“A higher probability exists that every member of your programming team will be attacked and killed by wolves in unrelated incidents on the same night.”

(part-1): plumbing

Wszystko to pliki

- katalog `.git` przechowuje:
  - commity
  - branche
  - tagi
  - i inne

Trees, blobs

- tree → przechowywanie katalogów
- blob → przechowywanie plików

> “[...] at the core of Git is a simple key-value data store.”

Object file

- git składa się z obietków
  - `/objects`
- są one skompresowane, ale `git cat-file`

Optymalizacje

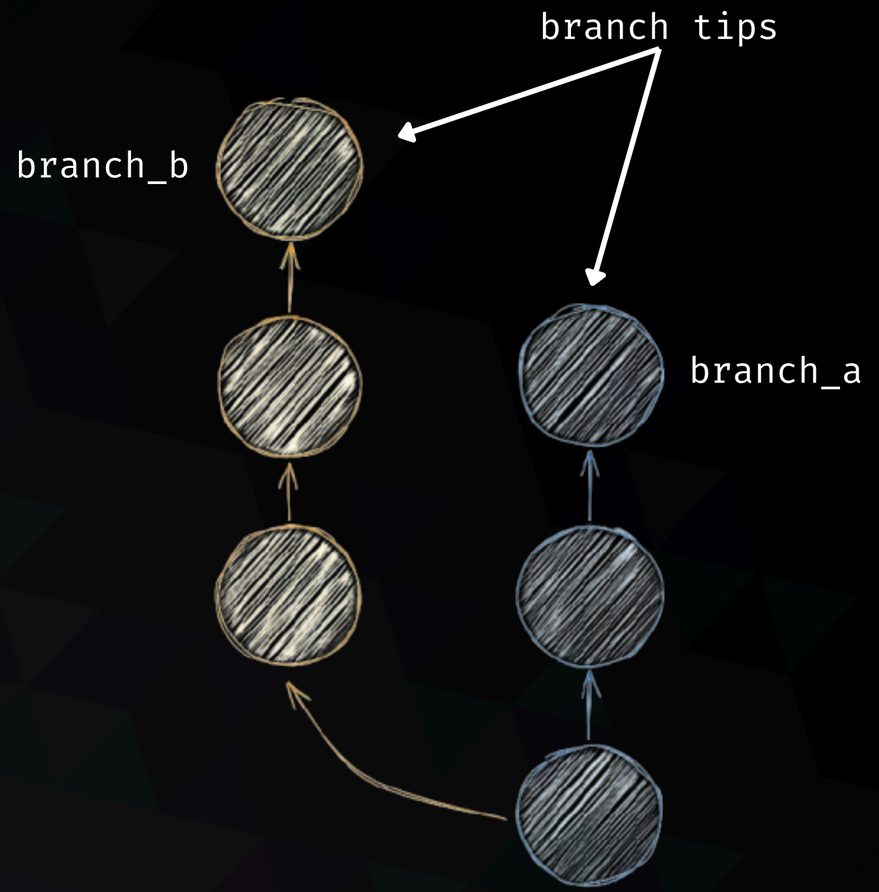
- git kompresuje i pakuje pliki
  - 50 lub 1MB
- deduplikuje pliki między commitami

(part-1): branch

Branch

- pozwala przechowywanie rożnych zmian jednocześnie i osobno
- jest to **wskaźnik** na dany commit
  - ten commit → “tip of the branch”
  - przez to lekkie
- wypisanie lokalnych branchy

```
git branch
```



Default branch

- nazwa to konwencja
- mozna ustawic domyślną w configu

(part-1): branch

Przydatne komendy

- stworzenie nowego brancha

```
git branch <branch_name>
```

- zmiana nazwy

```
git branch -m <old> <new>
```

- zmiana brancha

```
git switch <branch_name>
```

lub

```
git checkout <branch_name>
```

- usunięcie brancha

```
git branch -d <branch_name>
```

Warto wiedzieć

- stworzenie nowego brancha ustawi aktualny commit jako korzeń (base)
- nowy branch + przejście na niego (bardzo częste)

```
git switch -c <branch_name>
```

lub

```
git checkout -b <branch_name>
```

(part-1): branch

Dekorowanie logów

```
git log --decorate=[no|short|full]
```

- jednolinijkowe (bardzo częste)

```
git log --oneline
```

- graf branchy (przyda się zaraz)

```
git log --oneline --graph --all
```

- graf branchy + rodzice commitów (przyda się zaraz)

```
git log --oneline --graph --all --parents
```

Branche to też pliki!

- `.git/refs/heads`

(part-1): merge

Dopełnienie branchowania

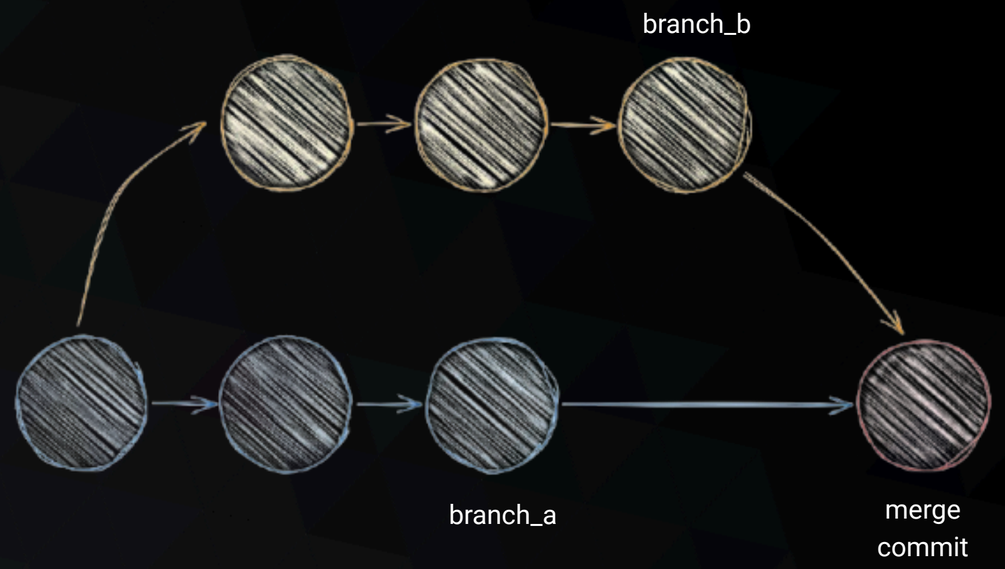
- pozwala na intergrację zmian (scalenie) między branchami
- merguje się “na” branch

```
git checkout branch_a
git merge branch_b
```

- jak to działa?
  a. szukanie przodka
  b. zaaplikowanie zmian z aktualnego brancha
  c. zaaplikowanie zmian z drugiego brancha
  d. zapisanie zmian jako nowy commit*

Fast-forward merge*

- jeżeli branch tip docelowego brancha się nie zmienił (=base)
- **nie tworzy** się nowy commit



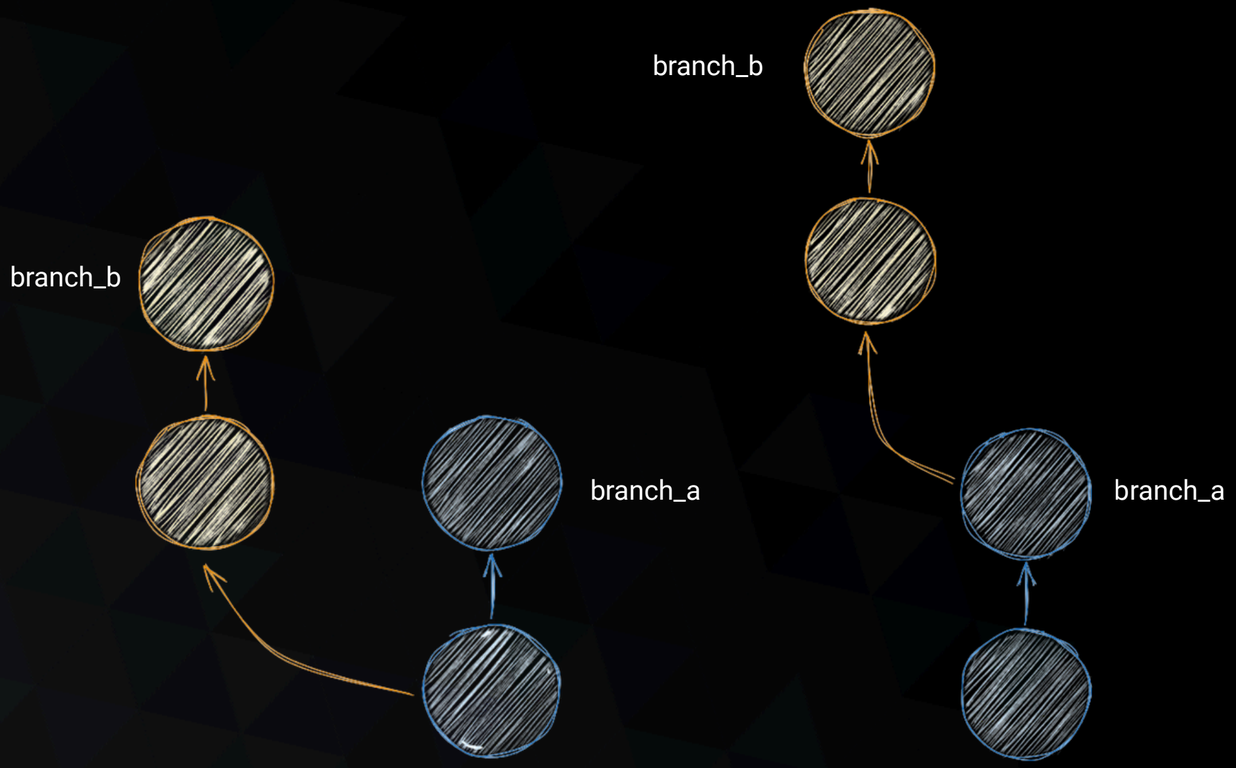
(part-1): rebase

Przepisywanie commitów?

- pozwala przepisać commity na tip innego brancha
- pozwala na utrzymanie liniowej historii (merge commit free)

```
git rebase [<branch>]
```

- użyteczne do “sprzątania” brancha (tryb interactive)
- **nie powinno** zastępować merge → to narzędzie o innym przeznaczeniu




```
git checkout branch_b
git rebase branch_a
```

(part-1): reset

Cofanie zmian

- pozwala cofnięcie ostatniego commita (lub wielu commitów)
  - commited → staged
  - staged → untracked

```
git reset --soft <SHA>
```

- pozwala na pełne cofnięcie do poprzedniego stanu (**bez zachowania zmian***)

```
git reset --hard <SHA>
```

(part-1): reflog-HEAD

Wskaźnik na teraz

- oznacza stan, gdzie jesteśmy “teraz”
- pozwala na łatwiejsze odwoływanie się do commitów (bez hashy)
  `HEAD~n`
  `HEAD^n` → n-ty przodek
- pozwala śledzenie ruchów*

  `HEAD@{n}` → n-ty ruch temu

Reflog

- wypisuje logi informujące, kiedy referencje i branch tips się zmieniały
- domyślnie przechowuje informacje przez 90 dni

  ```
  git reflog
  ```

part-2

remote

(part-2): remote

Źródło prawdy

- rozproszoność
- remote to tak naprawdę po prostu inne repozytorium
- gdy traktuje się je jako “źródło prawdy”, to według konwencji nazywa się `origin`

```
git remote add <name> <URL>
```

- samo dodanie **nie pobiera** zawartości
  - aby pobrać `objects, refs`

```
git fetch
```

- żeby zsynchronizować pliki, wystarczy merge

```
git merge origin/master
```

(part-2): GitHub

Git != GitHub

- GitHub to serwis hostujący repozytoria, a git to rozproszony system kontroli wersji

Po co?

- kopia zapasowa lokalnego repozytorium
- “źródło prawdy” dla lokalnego repozytorium
- łatwa kolaboracja z innymi
- portfolio
- i wiele więcej...

Alternatywy?

- Gitlab
- BitBucket
- Gitea
- self-hosting
- i wiele innych



GitHubCLI

- pozwala na intergrację z GitHubem w terminalu
- całkiem przydatne

```
gh <required> [optional]
```

(part-2): push-pull-pr

Push

- pozwala na “wypchnięcie” aktualnych zmian na remote
- trzeba być zautoryzowanym

```
git push <remote> <branch>
```

- można wypchnąć ze zmianą nazwy

```
git push <remote> <l_branch>:<r_branch>
```

- domyślnie **nie pozwoli** zmodyfikować brancha jeżeli nie jest przodkiem wypychanego commita
  - ale

```
git push -f
```

(part-2): push-pull-pr

Pull

- pozwala na “zaciągnięcie” zmian z remote na lokalne repozytorium
- trzeba być zautoryzowanym

```
git pull <remote> <branch>
```

- operuje w kilku trybach
  - `git pull --ff-only` → fast-forward (default)
  - `git pull --rebase` → rebase
  - `git pull --no-rebase` → merge
  - `git pull --squash` → `merge --squash`
- domyślną strategię można zmienić w configu, np.:

```
git config set pull.rebase false
```

(part-2): push-pull-pr

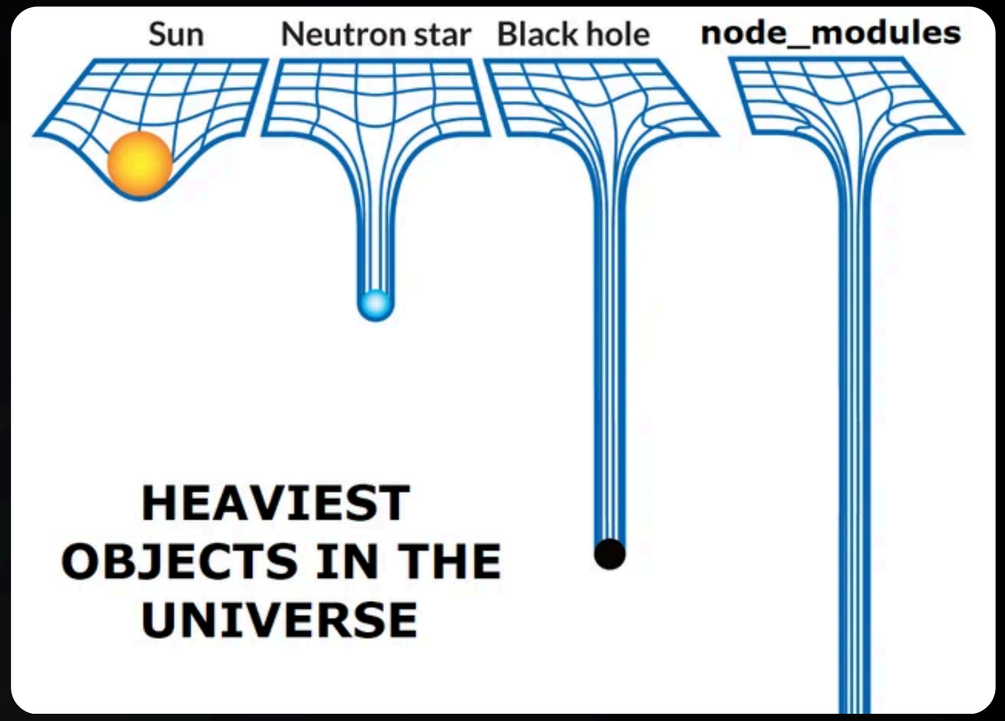
Pull request

- rzecz typowo związana z serwisem
- “prośba” o włączenie zaproponowanych zmian do remote
- feature_branch → master
- pozwala na tzw. **review** zmian
- umożliwia wiele intergracji np.:
  - automatyczne testy
  - review od AI
  - i inne
- typowo obowiązkowy krok przy wprowadzaniu jakichkolwiek zmian przy projektach zespołowych
- reviewer może odrzucić i wymusić zmiany, jeżeli nie spełniają *wymogów*
- gdy jest *approved* (+inne bramki kontroli), to jest mergowany, a branch jest usuwany

(part-2): .gitignore

Ignorowanie plików

- niektóre pliki nie powinny znajdować się w remote np.:
  - są zbędne (dependencies)
  - to niebezpieczne (np. klucze API, pass tokeny)
  - są lokalne (dev-specific)



(part-2): .gitignore

Wzorce ścieżek

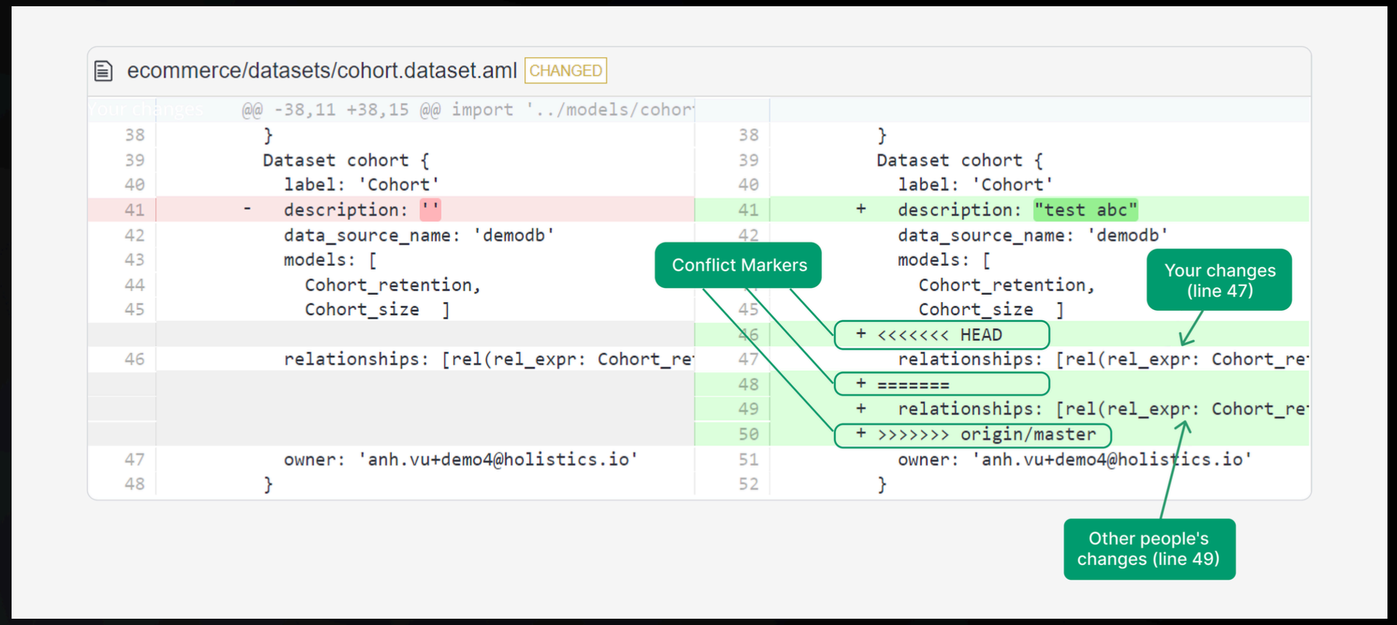
- “*” → dowolna liczba znaków poza “/”
- “/” → separator katalogów
- “!” → negacja
- “#” → komentarz
- kolejność ma znaczenie!

```
*.py
/src      data/     data/csv
!src/hello.txt
# this is a comment
```

(part-2): conflicts

To jest najgorsze

- w przypadku gdy integracja zmian wpływa na ten sam plik, może dojść do konfliktu
- git oznacza ten konflikt i prosi o rozwiązanie i dodanie do indexu
  - merge → raz
  - rebase → potencjalnie kilka razy
- górna sekcja (HEAD), to nasza wersja, dolna to “ich”



(part-2): misc

Fork

- zapewniane przez serwisy
- pozwala na stworzenie prywatnej kopii innego repozytorium
- częste przy.:
  - eksperymentowaniu
  - dostosowywaniu czegoś pod siebie
  - nadbudowywaniu

Stash

- pozwala na zapis aktualnych zmian do “schowka”
- działa jak stos
  - można dodawać i zdejmować

```
git stash
git stash pop
git stash apply
git stash drop
```

- można używać składni `stash@{n}`
  - n-ty stash

(part-2): misc

Przydatne narzędzia

- Visual Studio Code
  - source control (wbudowany)
  - GitLens → interaktywny rebase
  - Git Graph
- JetBrains IDE → mają dużo out-of-the-box
- TUI → lazygit
- GitHub CLI
- GitHub Desktop
- GitKraken

Opinie

- commit message
- rebase vs merge

Jak się nauczyć?

- praktyka
- Boot.dev → inspiracja tego wykładu
- Git Immersion
- Learn Git Branching
- Git Exercises

```
feat!: finish lecture
```